Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 8

Дисциплина МДК 03.02

Устранение уязвимостей в API NET 8 и тестирование безопасности

Вариант № 2
АИС «Умный кампус»
Модуль Library System

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3

1.1 Исходные уязвимости.....3

1.2 Реализация мер.....3

1.3 Проверка устранения.....4

1.4 Выводы.....5

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....7

ПРИЛОЖЕНИЕ А Листинг исходного кода.....8

ПРИЛОЖЕНИЕ Б Интеграционные проверки безопасности.....16

1 ВЫПОЛНЕНИЕ РАБОТЫ

Я выполнил работу по варианту 2 Library System АИС «Умный кампус» на C# и .NET 8. Модуль принимает PDF/EPUB/ZIP в карантин; права Reader и Librarian разделены [2]. Оформление выполнено по положению института [1].

1.1 Исходные уязвимости

В варианте 2 я исследовал Zip Slip и Unrestricted File Upload. LegacyUploadController объединял Root с entry.FullName без проверки, сохранял произвольные байты и разрешал запись по клиентской строке role. Пример before изолирован и не обслуживает HTTP-маршруты.

Белый список расширений не исправляет выход пути из каталога, а MIME запроса не доказывает тип содержимого. Я применил независимые проверки имён, байтов, прав, размеров и места хранения по рекомендациям OWASP [6].

Сведения представлены в таблице 1.

Таблица 1 — Сравнение подходов

Область	Уязвимый образец	Исправленный API
Путь ZIP	Path.Combine с entry.FullName	Проверка сегментов, ссылок и UUID
Формат	Любые байты	PDF/EPUB и структура архива
Права	Клиентская строка role	Серверный principal, LibrarianOnly
Поля	Смешаны с записью	FluentValidation до сохранения
База данных	Нет выделенной модели	LINQ EF Core с параметрами
Ресурсы	Нет ограничений	Размеры, распаковка, 3 запроса/мин
Выдача	Недоверенный файл сразу записан	Приватный карантин без публикации

Источник: составлено автором по результатам работы

1.2 Реализация мер

Authorize с LibrarianOnly получает роль из серверного principal. Reader получает 403, без действительного токена — 401. Лабораторная выдача

случайных токенов разрешена только Development с loopback и скрыта в Production [4].

MetadataValidator проверяет название, имя и длину. FilePolicy контролирует реально прочитанные байты, сигнатуру PDF и базовую структуру EPUB; DTD запрещён, XmlResolver отсутствует. Для ZIP отклоняются опасные сегменты, абсолютные пути, ссылки, дубли, более 12 записей, 3 МиБ распаковки и коэффициент сжатия выше 200. Книга ограничена 1 МиБ, входной файл — 2 МиБ; транспортный multipart имеет ещё 64 КиБ.

EF Core получает критерий поиска как параметр LINQ, а не часть собранного SQL. StoredName формируется из UUID. Все книги набора проверяются до сохранения; метаданные записываются в транзакции, при ошибке уже созданные файлы удаляются. Это компенсация файловых операций, а не распределённая транзакция БД и файловой системы [3].

Rate Limiting допускает три загрузки пользователя в минуту без очереди; четвёртая возвращает 429. Лимит распаковки защищает от другой нагрузки. Файловая выдача требует Published; новая загрузка всегда получает Quarantined [5].

1.3 Проверка устранения

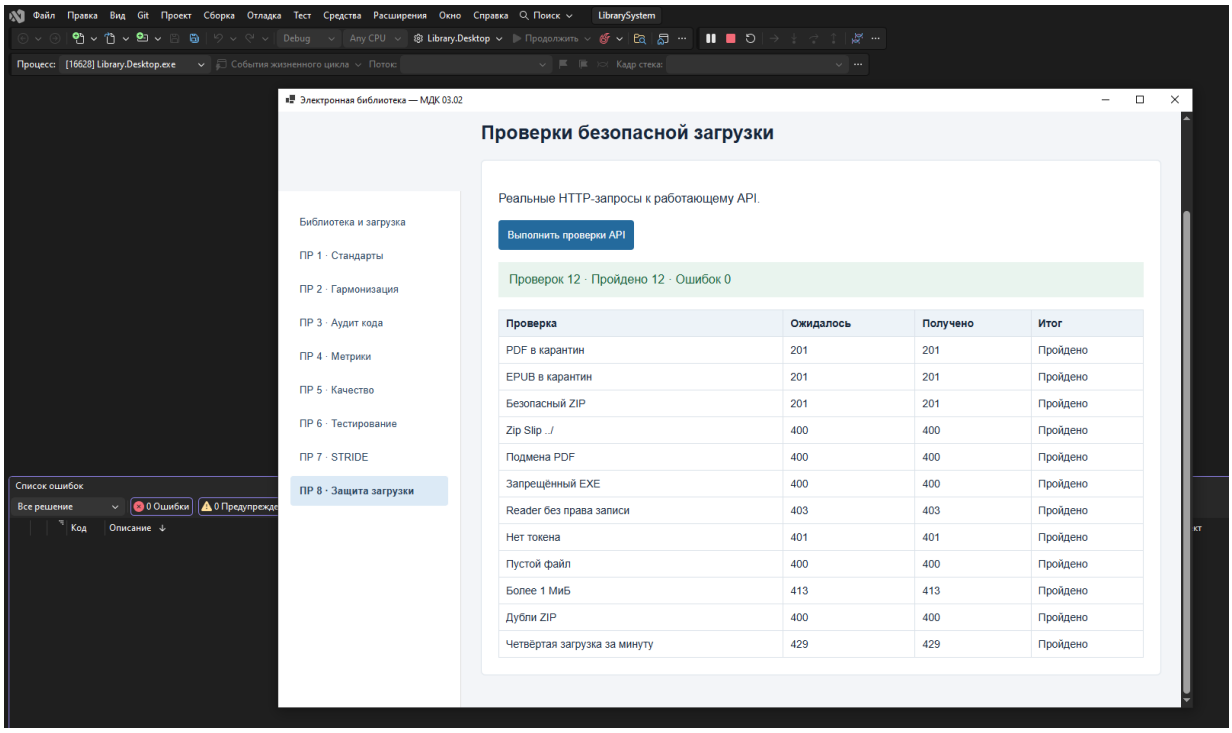
ZipSlipLeavesNoFiles проверяет пять опасных путей и отсутствие файлов; SpoofedPdfRejected — подмену PDF; ReaderCannotUpload — 403; FourthUploadRateLimited — 429. Общий прогон: 47 Passed, 0 Failed, 0 Skipped. Все имена и исходный журнал сохранены в verification.

Через интерфейс выполнены 12 реальных HTTP-сценариев: корректные форматы, ZIP, обход пути, подмена, запрещённый EXE, роль, токен, пустой файл, превышение размера, дубль и частота. Ответы совпали с ожидаемыми. Повторный прогон запускается кнопкой «Выполнить проверки API» на вкладке ПР 8.

Ручной сценарий: загрузить образец PDF с ролью Librarian и проверить HTTP 201 и статус Quarantined. Повторить загрузку с ролью Reader и проверить HTTP 403. Доступ к файлу карантина отдельно проверяется интеграционным тестом.

Сравнительный листинг обработчиков и всех мер приведён в приложении А; полный код интеграционных доказательств — в приложении Б. Базовая сигнатура PDF не заменяет антивирус, поэтому карантин не снимается автоматически.

На рисунке 1 приведён дополнительный сквозной прогон: все 12 HTTP-проверок совпали с ожидаемыми результатами. Интегрированные тесты xUnit выполняются отдельно; их листинг приведён в приложении Б.



Источник: снимок прогона на Windows, предоставленный автором.

Рисунок 1 — Результаты сквозных HTTP-проверок безопасной загрузки

1.4 Выводы

Я устранил доверие к архивному пути и произвольному содержимому, добавил FluentValidation, серверную политику, параметризацию EF Core и Rate Limiting. Автоматические и сквозные проверки подтвердили ожидаемое поведение исправленного API.

Полные исходники, входные данные и журналы переданы с отчётом. Основной листинг приведён в приложении А.

Исходный код и порядок запуска: репозиторий MDK0302-LibrarySystem.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Положение об оформлении письменных работ. Санкт-Петербургский институт экономики и управления. Методические материалы [1].

2. Практические работы МДК 03.02. Задания ПР 1–8 и эссе. Вариант 2 Library System. Материалы преподавателя [2].

3. Microsoft. EF Core. SQL queries and parameterization. — URL: https://learn.microsoft.com/en-us/ef/core/querying/sql-queries (дата обращения: 08.11.2026) [3].

4. Microsoft. Policy-based authorization in ASP.NET Core. — URL: https://learn.microsoft.com/en-us/aspnet/core/security/authorization/policies?view=aspnetcore-8.0 (дата обращения: 08.11.2026) [4].

5. Microsoft. Rate limiting middleware in ASP.NET Core. — URL: https://learn.microsoft.com/en-us/aspnet/core/performance/rate-limit?view=aspnetcore-8.0 (дата обращения: 08.11.2026) [5].

6. OWASP. File Upload Cheat Sheet. — URL: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html (дата обращения: 08.11.2026) [6].

ПРИЛОЖЕНИЕ А
Листинг исходного кода

before/LegacyUploadController.cs

```
using System.IO.Compression;
namespace Library.Before;
// Изолированный исходный образец. Не включается в сборку API.
public class LegacyUploadController
{
   private string Root = "App_Data";
   public string upload(string role, string name, byte[] bytes)
   {
      if (role == "Librarian" || role == "Reader")
      {
         if (bytes.Length == 0) return "error";
         if (name.EndsWith(".zip"))
         {
            using var zip = new ZipArchive(new MemoryStream(bytes));
            foreach (var entry in zip.Entries)
            {
               string path = Path.Combine(Root, entry.FullName);
               Directory.CreateDirectory(Path.GetDirectoryName(path)!);
               using var input = entry.Open();
               using var output = File.Create(path);
               input.CopyTo(output);
            }
         }
         else File.WriteAllBytes(Path.Combine(Root, name), bytes);
         return "ok";
      }
      return "error";
   }
   public int count() => Directory.GetFiles(Root).Length;
}
```

src/Library.Api/UploadController.cs

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.RateLimiting;
namespace Library.Api;


/// <summary>Принимает книги только от библиотекаря.</summary>
[ApiController]
[Route("api/books")]
[Authorize(Policy = "LibrarianOnly")]
public sealed class UploadController(UploadService service) : ControllerBase
{
   /// <summary>Создаёт записи карантина и возвращает 201.</summary>
   [HttpPost("upload")]
   [EnableRateLimiting("uploads")]
   [RequestSizeLimit(2 * 1024 * 1024 + 65536)]
   public async Task<IActionResult> Upload([FromForm] string title, IFormFile file, CancellationToken ct)
   {
      string actor = User.FindFirstValue(ClaimTypes.NameIdentifier)!;
```

```
        var rows = await service.SaveAsync(title, file, actor, ct);
        return StatusCode(201, rows.Select(x => new { x.Id, x.Title, x.OriginalName,
            x.Length, x.Sha256, Status = "Quarantined" }));
    }
}
```

src/Library.Core/FilePolicy.cs

```
using System.IO.Compression;
using System.Text;
using System.Xml;
using System.Xml.Linq;
namespace Library.Core;

public sealed record AcceptedFile(string OriginalName, byte[] Bytes, string Extension);
public sealed class RejectedFile(string message, int status = 400) : Exception(message)
{
    public int Status { get; } = status;
}

/// <summary>Проверяет формат и пути до записи недоверенного содержимого.</summary>
public static class FilePolicy
{
    public const int BookLimit = 1024 * 1024;
    public const int InputLimit = 2 * BookLimit;
    public const int InflatedLimit = 3 * BookLimit;
    public const int EntryLimit = 12;

    /// <summary>Проверяет сегменты пути с учётом имён Windows.</summary>
    public static bool IsSafeName(string name)
    {
        if (string.IsNullOrWhiteSpace(name) || name.Length > 200 ||
            name.Any(c => char.IsControl(c) || c is ':' or '"' or '*' or '?' or '<' or '>' or '|'))
            return false;
        string normalized = name.Replace('\\', '/');
        if (normalized.StartsWith('/')) return false;
        foreach (string part in normalized.Split('/'))
        {
            if (part is "" or "." or ".." || part.EndsWith('.') || part.EndsWith(' ')) return false;
            string stem = part.Split('.')[0].ToUpperInvariant();
            if (new[] { "CON", "PRN", "AUX", "NUL", "COM1", "LPT1" }.Contains(stem)) return false;
            if (stem.Length == 4 && (stem.StartsWith("COM") || stem.StartsWith("LPT")) &&
                stem[3] is >= '1' and <= '9') return false;
        }
        return true;
    }

    /// <summary>Читает поток с ограничением фактически полученных байтов.</summary>
    public static byte[] ReadLimited(Stream stream, int limit)
    {
        using var output = new MemoryStream();
        byte[] buffer = new byte[8192];
        int count;
        while ((count = stream.Read(buffer)) != 0)
        {
            if (output.Length + count > limit) throw new RejectedFile("Превышен размер файла", 413);
            output.Write(buffer, 0, count);
        }
        return output.ToArray();
```

```
}

/// <summary>Возвращает проверенный набор; сохранение выполняется отдельно.</summary>
public static AcceptedFile[] Validate(string filename, byte[] bytes)
{
   if (!IsSafeName(filename)) throw new RejectedFile("Небезопасное имя файла");
   if (bytes.Length == 0) throw new RejectedFile("Пустой файл");
   if (bytes.Length > InputLimit) throw new RejectedFile("Входной файл больше 2 МиБ", 413);
   if (Path.GetExtension(filename).Equals(".zip", StringComparison.OrdinalIgnoreCase))
      return ValidateZip(bytes);
   return [ValidateBook(filename, bytes)];
}

private static AcceptedFile ValidateBook(string name, byte[] bytes)
{
   if (bytes.Length == 0) throw new RejectedFile("Пустая книга");
   if (bytes.Length > BookLimit) throw new RejectedFile("Книга больше 1 МиБ", 413);
   string extension = Path.GetExtension(name).ToLowerInvariant();
   if (extension == ".pdf")
   {
      if (!bytes.AsSpan().StartsWith("%PDF-"u8) ||
         !Encoding.ASCII.GetString(bytes.AsSpan(Math.Max(0, bytes.Length - 1024))).Contains("%%EOF"))
         throw new RejectedFile("Сигнатура PDF не соответствует расширению");
   }
   else if (extension == ".epub") ValidateEpub(bytes);
   else throw new RejectedFile("Разрешены PDF, EPUB и ZIP-пакеты");
   return new(name, bytes, extension);
}

private static AcceptedFile[] ValidateZip(byte[] bytes)
{
   try
   {
      using var zip = new ZipArchive(new MemoryStream(bytes));
      if (zip.Entries.Count is 0 or > EntryLimit) throw new RejectedFile("От 1 до 12 записей ZIP");
      var names = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
      var files = new List<AcceptedFile>();
      long total = 0;
      foreach (var entry in zip.Entries)
      {
         if (!IsSafeName(entry.FullName) || !names.Add(entry.FullName.Replace('\\', '/')) ||
            ((entry.ExternalAttributes >> 16) & 0xF000) == 0xA000)
            throw new RejectedFile("Небезопасный путь, ссылка или дубль ZIP");
         if (entry.Length > BookLimit) throw new RejectedFile("Книга больше 1 МиБ", 413);
         if (entry.Length > 0 && (double)entry.Length / Math.Max(1, entry.CompressedLength) > 200)
            throw new RejectedFile("Коэффициент сжатия больше 200");
         using var stream = entry.Open();
         byte[] content = ReadLimited(stream, BookLimit);
         total += content.Length;
         if (total > InflatedLimit) throw new RejectedFile("Объём распаковки больше 3 МиБ", 413);
         files.Add(ValidateBook(entry.FullName, content));
      }
      return files.ToArray();
   }
   catch (InvalidDataException) { throw new RejectedFile("Повреждённый ZIP"); }
}

private static void ValidateEpub(byte[] bytes)
```

```
  {
    try
    {
      using var zip = new ZipArchive(new MemoryStream(bytes));
      if (zip.Entries.Count > EntryLimit) throw new RejectedFile("Слишком много записей EPUB");
      var names = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
      long total = 0;
      foreach (var e in zip.Entries)
      {
        if (!IsSafeName(e.FullName) || !names.Add(e.FullName) ||
          ((e.ExternalAttributes >> 16) & 0xF000) == 0xA000)
          throw new RejectedFile("Небезопасная структура EPUB");
        using var input = e.Open();
        total += ReadLimited(input, BookLimit).Length;
        if (total > InflatedLimit) throw new RejectedFile("Большой EPUB", 413);
      }
      var mime = zip.GetEntry("mimetype") ?? throw new RejectedFile("Нет mimetype EPUB");
      using var mimeReader = new StreamReader(mime.Open());
      if (mimeReader.ReadToEnd() != "application/epub+zip") throw new RejectedFile("Неверный
mimetype");
      var container = zip.GetEntry("META-INF/container.xml") ?? throw new RejectedFile("Нет
container.xml");
      using var xml = XmlReader.Create(container.Open(), new XmlReaderSettings
      { DtdProcessing = DtdProcessing.Prohibit, XmlResolver = null, MaxCharactersInDocument = 8192 });
      var doc = XDocument.Load(xml);
      string? package = doc.Descendants().FirstOrDefault(x => x.Name.LocalName == "rootfile")?
        .Attribute("full-path")?.Value;
      if (package is null || !IsSafeName(package) || zip.GetEntry(package) is null)
        throw new RejectedFile("Нет пакета EPUB");
    }
    catch (InvalidDataException) { throw new RejectedFile("Повреждённый EPUB"); }
    catch (XmlException) { throw new RejectedFile("Небезопасный XML EPUB"); }
  }
}
```

src/Library.Api/Storage.cs

```
using System.Security.Cryptography;
using FluentValidation;
using Library.Core;
using Microsoft.EntityFrameworkCore;
namespace Library.Api;


public sealed class Book
{
  public Guid Id { get; set; }
  public string Title { get; set; } = "";
  public string OriginalName { get; set; } = "";
  public string StoredName { get; set; } = "";
  public string Owner { get; set; } = "";
  public string Sha256 { get; set; } = "";
  public long Length { get; set; }
  public bool Published { get; set; }
}
public sealed class LibraryDb(DbContextOptions<LibraryDb> options) : DbContext(options)
{
  public DbSet<Book> Books => Set<Book>();
}
public sealed record UploadMetadata(string Title, string Filename, long Length);
/// <summary>Проверяет поля перед чтением и сохранением содержимого.</summary>
```

```
public sealed class MetadataValidator : AbstractValidator<UploadMetadata>
{
   public MetadataValidator()
   {
      RuleFor(x => x.Title).NotEmpty().MaximumLength(100)
         .Must(x => x is not null && !x.Any(char.IsControl));
      RuleFor(x => x.Filename).Must(FilePolicy.IsSafeName);
      RuleFor(x => x.Length).GreaterThan(0).LessThanOrEqualTo(FilePolicy.InputLimit);
   }
}
public sealed record StorageRoot(string Path);
/// <summary>Сохраняет только полностью проверенный набор в приватном карантине.</summary>
public sealed class UploadService(LibraryDb db, StorageRoot root, MetadataValidator validator)
{
   /// <summary>Выполняет валидацию, транзакцию метаданных и компенсацию файловой
записи.</summary>
   public async Task<Book[]> SaveAsync(string title, IFormFile file, string actor, CancellationToken ct)
   {
      var check = validator.Validate(new UploadMetadata(title, file.FileName, file.Length));
      if (!check.IsValid) throw new RejectedFile("Некорректные поля загрузки");
      using var input = file.OpenReadStream();
      var accepted = FilePolicy.Validate(file.FileName, FilePolicy.ReadLimited(input, FilePolicy.InputLimit));
      var rows = accepted.Select(x => new Book { Id = Guid.NewGuid(), Title = title,
         OriginalName = x.OriginalName, Owner = actor, StoredName = Guid.NewGuid().ToString("N") +
x.Extension,
         Length = x.Bytes.Length, Sha256 = Convert.ToHexString(SHA256.HashData(x.Bytes)), Published =
false }).ToArray();
      var written = new List<string>();
      await using var tx = await db.Database.BeginTransactionAsync(ct);
      try
      {
         for (int i = 0; i < rows.Length; i++)
         {
            string path = System.IO.Path.Combine(root.Path, "quarantine", rows[i].StoredName);
            await using var output = new FileStream(path, FileMode.CreateNew, FileAccess.Write);
            written.Add(path);
            await output.WriteAsync(accepted[i].Bytes, ct);
         }
         db.Books.AddRange(rows);
         await db.SaveChangesAsync(ct);
         await tx.CommitAsync(ct);
         return rows;
      }
      catch
      {
         await tx.RollbackAsync(CancellationToken.None);
         foreach (string path in written) File.Delete(path);
         throw;
      }
   }
}
```

src/Library.Api/Security.cs

```
using System.Collections.Concurrent;
using System.Security.Claims;
using System.Security.Cryptography;
using System.Text.Encodings.Web;
using Microsoft.AspNetCore.Authentication;
using Microsoft.Extensions.Options;
namespace Library.Api;
```

```
/// <summary>Случайные серверные токены стенда без клиентского назначения роли.</summary>
public sealed class TokenRegistry
{
   private readonly ConcurrentDictionary<string, ClaimsPrincipal> _tokens = new();
   public string Issue(string role, string actor)
   {
      string token = Convert.ToHexString(RandomNumberGenerator.GetBytes(32));
      _tokens[token] = new ClaimsPrincipal(new ClaimsIdentity(new[] {
         new Claim(ClaimTypes.NameIdentifier, actor), new Claim(ClaimTypes.Role, role) }, "LabToken"));
      return token;
   }
   public ClaimsPrincipal? Find(string token) => _tokens.GetValueOrDefault(token);
}
public sealed class TokenHandler(IOptionsMonitor<AuthenticationSchemeOptions> options,
   ILoggerFactory logger, UrlEncoder encoder, TokenRegistry tokens)
   : AuthenticationHandler<AuthenticationSchemeOptions>(options, logger, encoder)
{
   protected override Task<AuthenticateResult> HandleAuthenticateAsync()
   {
      string value = Request.Headers.Authorization.ToString();
      var principal = value.StartsWith("Bearer ") ? tokens.Find(value[7..]) : null;
      return Task.FromResult(principal is null ? AuthenticateResult.NoResult() :
         AuthenticateResult.Success(new AuthenticationTicket(principal, Scheme.Name)));
   }
}
```

src/Library.Api/BooksController.cs

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
namespace Library.Api;

/// <summary>Читает каталог с параметризованным LINQ и изоляцией карантина.</summary>
[ApiController]
[Route("api/books")]
[Authorize]
public sealed class BooksController(LibraryDb db, StorageRoot root) : ControllerBase
{
   /// <summary>Не раскрывает чужие неопубликованные документы.</summary>
   [HttpGet]
   public async Task<IActionResult> Get([FromQuery] string? query, CancellationToken ct)
   {
      string actor = User.FindFirstValue(ClaimTypes.NameIdentifier)!;
      bool librarian = User.IsInRole("Librarian");
      var books = db.Books.Where(x => x.Published || (librarian && x.Owner == actor));
      if (!string.IsNullOrWhiteSpace(query)) books = books.Where(x => x.Title.Contains(query));
      return Ok(await books.Select(x => new { x.Id, x.Title, x.OriginalName, x.Length,
         Status = x.Published ? "Published" : "Quarantined" }).ToArrayAsync(ct));
   }
   /// <summary>Выдаёт только опубликованный образец вне веб-каталога.</summary>
   [HttpGet("{id:guid}/file")]
   public async Task<IActionResult> Download(Guid id, CancellationToken ct)
   {
      var book = await db.Books.SingleOrDefaultAsync(x => x.Id == id && x.Published, ct);
      if (book is null) return NotFound();
```

```
        Response.Headers["X-Content-Type-Options"] = "nosniff";
        return PhysicalFile(Path.Combine(root.Path, "published", book.StoredName),
            "application/octet-stream", book.OriginalName);
    }
}
```

src/Library.Api/Program.cs

```
using System.Net;
using System.Security.Claims;
using System.Text.Json;
using System.Threading.RateLimiting;
using Library.Api;
using Library.Core;
using Microsoft.AspNetCore.Authentication;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);
string storage = builder.Configuration["StorageRoot"] ??
Path.Combine(builder.Environment.ContentRootPath, "App_Data");
Directory.CreateDirectory(storage);
Directory.CreateDirectory(Path.Combine(storage, "quarantine"));
Directory.CreateDirectory(Path.Combine(storage, "published"));
builder.Services.AddSingleton<StorageRoot>(services => new StorageRoot(
    services.GetRequiredService<IConfiguration>()["StorageRoot"] ??
Path.Combine(builder.Environment.ContentRootPath, "App_Data")));
builder.Services.AddSingleton<TokenRegistry>();
builder.Services.AddSingleton<MetadataValidator>();
builder.Services.AddScoped<UploadService>();
builder.Services.AddDbContext<LibraryDb>(o => o.UseSqlite("Data Source=" + Path.Combine(storage,
"library.db")));
builder.Services.AddControllers();
builder.Services.AddAuthentication("LabToken").AddScheme<AuthenticationSchemeOptions,
TokenHandler>("LabToken", null);
builder.Services.AddAuthorization(o => o.AddPolicy("LibrarianOnly", p =>
p.RequireAuthenticatedUser().RequireRole("Librarian")));
builder.Services.AddRateLimiter(o => {
    o.RejectionStatusCode = 429;
    o.AddPolicy("uploads", context => RateLimitPartition.GetFixedWindowLimiter(
        context.User.FindFirstValue(ClaimTypes.NameIdentifier) ?? "anonymous", _ => new
FixedWindowRateLimiterOptions
        { PermitLimit = 3, Window = TimeSpan.FromMinutes(1), QueueLimit = 0 }));
});
var app = builder.Build();
storage = app.Services.GetRequiredService<StorageRoot>().Path;
Directory.CreateDirectory(storage);
Directory.CreateDirectory(Path.Combine(storage, "quarantine"));
Directory.CreateDirectory(Path.Combine(storage, "published"));
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<LibraryDb>();
    db.Database.EnsureCreated();
    if (!db.Books.Any())
    {
        byte[] sample = Samples.Pdf();
        File.WriteAllBytes(Path.Combine(storage, "published", "sample.pdf"), sample);
        db.Books.Add(new Book { Id = Guid.Parse("11111111-1111-4111-8111-111111111111"), Title = "Учебный
документ библиотеки",
            OriginalName = "sample.pdf", StoredName = "sample.pdf", Owner = "system", Published = true, Length
= sample.Length });
```

```
        db.SaveChanges();
    }
}
var auditLock = new object();
app.Use(async (context, next) => {
    try { await next(); }
    catch (RejectedFile error) {
        context.Response.StatusCode = error.Status;
        await context.Response.WriteAsJsonAsync(new { code = "upload_rejected", message = error.Message });
    }
    catch (Exception error) {
        app.Logger.LogError(error, "Ошибка запроса {RequestId}", context.TraceIdentifier);
        context.Response.StatusCode = 500;
        await context.Response.WriteAsJsonAsync(new { code = "server_error", requestId =
context.TraceIdentifier });
    }
    finally {
        if (context.Request.Path == "/api/books/upload") lock (auditLock)
            File.AppendAllText(Path.Combine(storage, "audit.jsonl"), JsonSerializer.Serialize(new {
                requestId = context.TraceIdentifier, actor =
context.User.FindFirstValue(ClaimTypes.NameIdentifier),
                status = context.Response.StatusCode, at = DateTimeOffset.UtcNow }) + Environment.NewLine);
    }
});
app.UseAuthentication();app.UseAuthorization();app.UseRateLimiter();
app.MapControllers();app.MapGet("/health", () => Results.Ok(new { status = "ok" }));
LabEndpoints.Map(app);
app.Run();
public partial class Program { }
```

ПРИЛОЖЕНИЕ Б
Интеграционные проверки безопасности

tests/Library.Tests/ApiHarness.cs

```
using Library.Api;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.TestHost;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
namespace Library.Tests;

public sealed class ApiHarness : IDisposable
{
   public string Root { get; } = Path.Combine(Path.GetTempPath(), "LibraryTests",
Guid.NewGuid().ToString("N"));
   public WebApplicationFactory<Program> Factory { get; }
   public HttpClient Client { get; }
   public ApiHarness(string environment = "Development")
   {
      Factory = new WebApplicationFactory<Program>().WithWebHostBuilder(builder => {
         builder.UseEnvironment(environment);
         builder.ConfigureAppConfiguration((_, cfg) => cfg.AddInMemoryCollection(new
Dictionary<string,string?> { ["StorageRoot"] = Root }));
      });
      Client = Factory.CreateClient();
   }
   public void Role(string role, string actor = "alice")
   {
      string token = Factory.Services.GetRequiredService<TokenRegistry>().Issue(role, actor);
      Client.DefaultRequestHeaders.Authorization = new("Bearer", token);
   }
   public async Task<HttpResponseMessage> Upload(byte[] bytes, string filename = "book.pdf", string title =
"Книга")
   {
      using var data = new MultipartFormDataContent();data.Add(new StringContent(title), "title");
      data.Add(new ByteArrayContent(bytes), "file", filename);
      return await Client.PostAsync("/api/books/upload", data);
   }
   public void Dispose()
   {
      Client.Dispose();Factory.Dispose();
      Microsoft.Data.Sqlite.SqliteConnection.ClearAllPools();
      if (Directory.Exists(Root)) Directory.Delete(Root, true);
   }
}
```

tests/Library.Tests/UploadTests.cs

```
using System.Net;
using System.Text;
using System.Text.Json;
using FluentAssertions;
using Library.Core;
using Xunit;
namespace Library.Tests;

public sealed class UploadTests
```

```
{
    [Fact]
    public async Task InternalFailureDoesNotLeakPathOrStack()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        string target = Path.Combine(h.Root, "quarantine");
        Directory.Delete(target);File.WriteAllText(target, "blocked");
        var response = await h.Upload(Samples.Pdf());
        response.StatusCode.Should().Be(HttpStatusCode.InternalServerError);
        string text = await response.Content.ReadAsStringAsync();
        text.Should().Contain("server_error").And.NotContain(h.Root).And.NotContain("StackTrace");
    }
    [Fact]
    public async Task LibrarianUploadsPdfToQuarantine()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        var r = await h.Upload(Samples.Pdf());r.StatusCode.Should().Be(HttpStatusCode.Created);
        string result = await r.Content.ReadAsStringAsync();result.Should().Contain("Quarantined");
        var files = Directory.GetFiles(Path.Combine(h.Root, "quarantine"));files.Should().HaveCount(1);
        Path.GetFileName(files[0]).Should().NotBe("book.pdf");
    }
    [Fact]
    public async Task LibrarianUploadsEpub()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload(Samples.Epub(), "book.epub")).StatusCode.Should().Be(HttpStatusCode.Created);
    }
    [Fact]
    public async Task SafeZipCreatesTwoBooks()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload(Samples.Zip(("a.pdf", Samples.Pdf()), ("b.epub", Samples.Epub())), "books.zip"))
            .StatusCode.Should().Be(HttpStatusCode.Created);
        Directory.GetFiles(Path.Combine(h.Root, "quarantine")).Should().HaveCount(2);
    }
    [Fact]
    public async Task ReaderCannotUpload()
    {
        using var h = new ApiHarness();h.Role("Reader");
        (await h.Upload(Samples.Pdf())).StatusCode.Should().Be(HttpStatusCode.Forbidden);
    }
    [Fact]
    public async Task MissingTokenRejected()
    {
        using var h = new ApiHarness();
        (await h.Upload(Samples.Pdf())).StatusCode.Should().Be(HttpStatusCode.Unauthorized);
    }
    [Fact]
    public async Task InvalidTokenRejected()
    {
        using var h = new ApiHarness();h.Client.DefaultRequestHeaders.Authorization = new("Bearer",
"invalid");
        (await h.Upload(Samples.Pdf())).StatusCode.Should().Be(HttpStatusCode.Unauthorized);
    }
    [Theory]
    [InlineData("../escape.pdf")]
    [InlineData("..\\escape.pdf")]
    [InlineData("/absolute.pdf")]
    [InlineData("C:\\escape.pdf")]
```

```
    [InlineData("a/../../escape.pdf")]
    public async Task ZipSlipLeavesNoFiles(string path)
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload(Samples.Zip((path, Samples.Pdf())),
"bad.zip")).StatusCode.Should().Be(HttpStatusCode.BadRequest);
        Directory.GetFiles(Path.Combine(h.Root, "quarantine")).Should().BeEmpty();
        File.Exists(Path.Combine(h.Root, "escape.pdf")).Should().BeFalse();
    }
    [Fact]
    public async Task SpoofedPdfRejected()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload("MZ executable"u8.ToArray())).StatusCode.Should().Be(HttpStatusCode.BadRequest);
    }
    [Fact]
    public async Task ExecutableRejected()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload("MZ"u8.ToArray(), "bad.exe")).StatusCode.Should().Be(HttpStatusCode.BadRequest);
    }
    [Fact]
    public async Task EmptyFileRejected()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload([])).StatusCode.Should().Be(HttpStatusCode.BadRequest);
    }
    [Fact]
    public async Task ExactOneMebibyteAccepted()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        byte[] bytes = new byte[FilePolicy.BookLimit];Samples.Pdf().CopyTo(bytes,0);"%
%EOF"u8.CopyTo(bytes.AsSpan(bytes.Length-5));
        (await h.Upload(bytes)).StatusCode.Should().Be(HttpStatusCode.Created);
    }
    [Fact]
    public async Task OneByteOverBookLimitRejected()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload(new
byte[FilePolicy.BookLimit+1])).StatusCode.Should().Be(HttpStatusCode.RequestEntityTooLarge);
    }
    [Fact]
    public async Task DuplicateNamesRejected()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload(Samples.Zip(("a.pdf", Samples.Pdf()), ("A.pdf", Samples.Pdf())), "bad.zip"))
            .StatusCode.Should().Be(HttpStatusCode.BadRequest);
    }
    [Fact]
    public async Task MixedZipSavesNothing()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload(Samples.Zip(("a.pdf", Samples.Pdf()), ("bad.exe", "MZ"u8.ToArray())), "bad.zip"))
            .StatusCode.Should().Be(HttpStatusCode.BadRequest);
        Directory.GetFiles(Path.Combine(h.Root, "quarantine")).Should().BeEmpty();
    }
    [Fact]
    public async Task FourthUploadRateLimited()
```

```
  {
    using var h = new ApiHarness();h.Role("Librarian");
    for (int i=0;i<3;i++) (await h.Upload(Samples.Pdf())).StatusCode.Should().Be(HttpStatusCode.Created);
    (await h.Upload(Samples.Pdf())).StatusCode.Should().Be(HttpStatusCode.TooManyRequests);
  }
  [Fact]
  public async Task OtherLibrarianCannotSeeQuarantine()
  {
    using var h = new ApiHarness();h.Role("Librarian", "alice");await h.Upload(Samples.Pdf(),title:"Private
title");
    h.Role("Librarian", "bob");
    (await h.Client.GetStringAsync("/api/books")).Should().NotContain("Private title");
  }
  [Fact]
  public async Task QuarantineCannotBeDownloaded()
  {
    using var h = new ApiHarness();h.Role("Librarian");var r=await h.Upload(Samples.Pdf());
    string id=JsonDocument.Parse(await
r.Content.ReadAsStringAsync()).RootElement[0].GetProperty("id").GetString()!;
    (await h.Client.GetAsync("/api/books/"+id+"/file")).StatusCode.Should().Be(HttpStatusCode.NotFound);
  }
  [Fact]
  public async Task ReaderDownloadsPublishedSample()
  {
    using var h = new ApiHarness();h.Role("Reader");
    var r=await h.Client.GetAsync("/api/books/11111111-1111-4111-8111-111111111111/file");
    r.StatusCode.Should().Be(HttpStatusCode.OK);r.Headers.GetValues("X-Content-Type-
Options").Should().Contain("nosniff");
  }
  [Fact]
  public async Task SqlLikeQueryRemainsData()
  {
    using var h = new ApiHarness();h.Role("Reader");
    var r=await h.Client.GetAsync("/api/books?query="+Uri.EscapeDataString("' OR 1=1 --"));
    r.StatusCode.Should().Be(HttpStatusCode.OK);(await r.Content.ReadAsStringAsync()).Should().Be("[]");
  }
  [Fact]
  public async Task ProductionHidesLabSession()
  {
    using var h = new ApiHarness("Production");
    (await h.Client.GetAsync("/lab/session/Librarian")).StatusCode.Should().Be(HttpStatusCode.NotFound);
  }
  [Fact]
  public async Task RejectedUploadIsAuditedWithoutToken()
  {
    using var h = new ApiHarness();h.Role("Reader");await h.Upload(Samples.Pdf());
    string log=File.ReadAllText(Path.Combine(h.Root,"audit.jsonl"));
    log.Should().Contain("403").And.Contain("alice").And.NotContain("Bearer");
  }
  [Fact]
  public async Task LongTitleRejected()
  {
    using var h = new ApiHarness();h.Role("Librarian");
    (await h.Upload(Samples.Pdf(),title:new
string('A',101))).StatusCode.Should().Be(HttpStatusCode.BadRequest);
  }
}
```

tests/Library.Tests/PolicyTests.cs

```
using System.IO.Compression;
```

```
using System.Text;
using FluentAssertions;
using Library.Core;
using Xunit;
namespace Library.Tests;
public sealed class PolicyTests
{
   [Theory]
   [InlineData("book.pdf",true)]
   [InlineData("books/a.epub",true)]
   [InlineData("../a.pdf",false)]
   [InlineData("C:\\a.pdf",false)]
   [InlineData("COM9.pdf",false)]
   [InlineData("a.pdf.",false)]
   [InlineData("a//b.pdf",false)]
   [InlineData("LPT3.pdf",false)]
   public void PathRules(string name,bool expected) => FilePolicy.IsSafeName(name).Should().Be(expected);
   [Fact]
   public void ThirteenEntriesRejected()
   {
      byte[] zip=Samples.Zip(Enumerable.Range(0,13).Select(i=>($"{i}.pdf",Samples.Pdf())).ToArray());
      Action act=()=>FilePolicy.Validate("bad.zip",zip);act.Should().Throw<RejectedFile>();
   }
   [Fact]
   public void TwelveEntriesAccepted()
   {
      byte[] zip=Samples.Zip(Enumerable.Range(0,12).Select(i=>($"{i}.pdf",Samples.Pdf())).ToArray());
      FilePolicy.Validate("good.zip",zip).Should().HaveCount(12);
   }
   [Fact]
   public void DamagedZipRejected()
   {
      Action
act=()=>FilePolicy.Validate("bad.zip","PKbroken"u8.ToArray());act.Should().Throw<RejectedFile>();
   }
   [Fact]
   public void StreamLimitCountsActualBytes()
   {
      Action act=()=>FilePolicy.ReadLimited(new MemoryStream(new byte[11]),10);
      act.Should().Throw<RejectedFile>().Which.Status.Should().Be(413);
   }
   [Fact]
   public void EpubWithoutMimetypeRejected()
   {
      Action act=()=>FilePolicy.Validate("bad.epub",Samples.Zip(("a.xhtml","text"u8.ToArray())));
      act.Should().Throw<RejectedFile>();
   }
   [Fact]
   public void EpubDtdRejected()
   {
      byte[] zip=Samples.Zip(("mimetype","application/epub+zip"u8.ToArray()),
         ("META-INF/container.xml",Encoding.UTF8.GetBytes("<!DOCTYPE x [<!ENTITY a SYSTEM
'file:///never'>]><container>&a;</container>")));
      Action act=()=>FilePolicy.Validate("bad.epub",zip);act.Should().Throw<RejectedFile>();
   }
   [Fact]
   public void SymbolicLinkRejected()
   {
      using var data=new MemoryStream();
```

```
        using(var zip=new ZipArchive(data,ZipArchiveMode.Create,true))
        {var e=zip.CreateEntry("link.pdf");e.ExternalAttributes=unchecked((int)0xA1FF0000);using var
s=e.Open();s.Write(Samples.Pdf());}
        Action act=()=>FilePolicy.Validate("link.zip",data.ToArray());act.Should().Throw<RejectedFile>();
    }
    [Fact]
    public void CompressionBombRejected()
    {
        using var data=new MemoryStream();
        using(var zip=new ZipArchive(data,ZipArchiveMode.Create,true))
        {using var s=zip.CreateEntry("big.pdf",CompressionLevel.SmallestSize).Open();s.Write(new
byte[500000]);}
        Action act=()=>FilePolicy.Validate("bomb.zip",data.ToArray());act.Should().Throw<RejectedFile>();
    }
}
```